In [1]:
```
import os, random
import numpy as np
from tqdm.auto import tqdm

SEED = 42
random.seed(SEED)
np.random.seed(SEED)

# InsightFace + onnxruntime
try:
    import insightface
    from insightface.app import FaceAnalysis
except Exception:
    !pip -q install insightface onnxruntime
    import insightface
    from insightface.app import FaceAnalysis

print("insightface:", insightface.__version__)
```

```
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 439.5/439.5 kB 8.8 MB/s eta 0:00:00a 0:
00:01
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 17.1/17.1 MB 84.7 MB/s eta 0:00:00:00:01
00:01
  Building wheel for insightface (pyproject.toml) ... done
insightface: 0.7.3
```

In [2]:
```
import cv2

app = FaceAnalysis(name="buffalo_l")
app.prepare(ctx_id=0, det_size=(640, 640))

def get_largest_face(faces):
    if not faces:
        return None
    areas = []
    for f in faces:
        x1, y1, x2, y2 = f.bbox.astype(int)
        areas.append((x2-x1) * (y2-y1))
    return faces[int(np.argmax(areas))]

def image_to_embedding(img_path):
    img = cv2.imread(img_path)
    if img is None:
        return None
    faces = app.get(img)
    face = get_largest_face(faces)
    if face is None:
        return None
    emb = face.embedding.astype(np.float32)
    emb /= (np.linalg.norm(emb) + 1e-12)
    return emb
```

```
download_path: /root/.insightface/models/buffalo_l
Downloading /root/.insightface/models/buffalo_l.zip from https://github.com/deepinsi
ght/insightface/releases/download/v0.7/buffalo_l.zip...
100%|██████████| 281857/281857 [00:19<00:00, 14637.52KB/s]
/usr/local/lib/python3.12/dist-packages/onnxruntime/capi/onnxruntime_inference_colle
ction.py:123: UserWarning: Specified provider 'CUDAExecutionProvider' is not in avai
lable provider names.Available providers: 'AzureExecutionProvider, CPUExecutionProvi
der'
  warnings.warn(
Applied providers: ['CPUExecutionProvider'], with options: {'CPUExecutionProvider':
{}}
find model: /root/.insightface/models/buffalo_l/1k3d68.onnx landmark_3d_68 ['None',
3, 192, 192] 0.0 1.0
Applied providers: ['CPUExecutionProvider'], with options: {'CPUExecutionProvider':
{}}
find model: /root/.insightface/models/buffalo_l/2d106det.onnx landmark_2d_106 ['Non
e', 3, 192, 192] 0.0 1.0
Applied providers: ['CPUExecutionProvider'], with options: {'CPUExecutionProvider':
{}}
find model: /root/.insightface/models/buffalo_l/det_10g.onnx detection [1, 3, '?',
'?'] 127.5 128.0
Applied providers: ['CPUExecutionProvider'], with options: {'CPUExecutionProvider':
{}}
find model: /root/.insightface/models/buffalo_l/genderage.onnx genderage ['None', 3,
96, 96] 0.0 1.0
Applied providers: ['CPUExecutionProvider'], with options: {'CPUExecutionProvider':
{}}
find model: /root/.insightface/models/buffalo_l/w600k_r50.onnx recognition ['None',
3, 112, 112] 127.5 127.5
set det-size: (640, 640)
```

In [10]:
```python
import os

LFW_ROOT = "/kaggle/input/datasets/jessicali9530/lfw-dataset/lfw-deepfunneled/lfw-d

# validation
if not os.path.isdir(LFW_ROOT):
    raise FileNotFoundError(f"LFW_ROOT not found: {LFW_ROOT}")

# Check identities
identities = [d for d in os.listdir(LFW_ROOT) if os.path.isdir(os.path.join(LFW_ROO

print("LFW_ROOT:", LFW_ROOT)
print("Number of identities:", len(identities))
print("Sample identities:", identities[:5])
```

```
LFW_ROOT: /kaggle/input/datasets/jessicali9530/lfw-dataset/lfw-deepfunneled/lfw-deep
funneled
Number of identities: 5749
Sample identities: ['Tyler_Hamilton', 'Bernard_Siegel', 'Blythe_Danner', 'Gene_Robin
son', 'Nicole_Parker']
```

In [11]:
```python
def list_identities(root):
    ids = []
    for name in os.listdir(root):
        d = os.path.join(root, name)
```

```
        if not os.path.isdir(d):
            continue
        imgs = [
            os.path.join(d, f)
            for f in os.listdir(d)
            if f.lower().endswith((".jpg", ".jpeg", ".png"))
        ]
        if len(imgs) >= 2:
            ids.append((name, sorted(imgs)))
    return ids

identities = list_identities(LFW_ROOT)
print("identities (>=2 imgs):", len(identities))

rng = np.random.default_rng(SEED)
rng.shuffle(identities)

n = len(identities)
n_train = int(0.70 * n)
n_val   = int(0.15 * n)

train_ids = identities[:n_train]
val_ids   = identities[n_train:n_train+n_val]
test_ids  = identities[n_train+n_val:]

print("train/val/test:", len(train_ids), len(val_ids), len(test_ids))
```

```
identities (>=2 imgs): 1680
train/val/test: 1176 252 252
```

In [12]:

```
def build_gallery(id_list, ref_index=0, max_ids=None):
    gallery = {}
    used = id_list if max_ids is None else id_list[:max_ids]
    skipped = 0

    for person, imgs in tqdm(used, desc="gallery"):
        ref_img = imgs[ref_index]
        emb = image_to_embedding(ref_img)
        if emb is None:
            skipped += 1
            continue
        gallery[person] = {"ref_img": ref_img, "ref_emb": emb}
    return gallery, skipped

MAX_TRAIN_IDS = 1200
MAX_VAL_IDS   = 400
MAX_TEST_IDS  = 400

train_gallery, s1 = build_gallery(train_ids, max_ids=MAX_TRAIN_IDS)
val_gallery,   s2 = build_gallery(val_ids,   max_ids=MAX_VAL_IDS)
test_gallery,  s3 = build_gallery(test_ids,  max_ids=MAX_TEST_IDS)

print("train_gallery:", len(train_gallery), "skipped:", s1)
print("val_gallery:", len(val_gallery), "skipped:", s2)
print("test_gallery:", len(test_gallery), "skipped:", s3)
```

```
gallery:   0%|          | 0/1176 [00:00<?, ?it/s]
```

```
gallery:   0%|          | 0/252 [00:00<?, ?it/s]
gallery:   0%|          | 0/252 [00:00<?, ?it/s]
train_gallery: 1172 skipped: 4
val_gallery: 251 skipped: 1
test_gallery: 252 skipped: 0
```

In [13]:
```python
def build_probes(id_list, gallery, ref_index=0, max_probes_per_id=3):
    probes = []
    rng = np.random.default_rng(SEED)
    for person, imgs in id_list:
        if person not in gallery:
            continue
        other_imgs = [p for i, p in enumerate(imgs) if i != ref_index]
        rng.shuffle(other_imgs)
        for p in other_imgs[:max_probes_per_id]:
            probes.append((person, p))
    return probes

val_probes  = build_probes(val_ids[:MAX_VAL_IDS],  val_gallery,  max_probes_per_id=
test_probes = build_probes(test_ids[:MAX_TEST_IDS], test_gallery, max_probes_per_id

print("val probes:", len(val_probes))
print("test probes:", len(test_probes))
```

```
val probes: 489
test probes: 479
```

In [14]:
```python
def cosine_sim(a, b):
    return float(np.dot(a, b))  # both are L2-normalized

def best_match(probe_emb, gallery):
    best_person = None
    best_score = -1.0
    for person, data in gallery.items():
        score = cosine_sim(probe_emb, data["ref_emb"])
        if score > best_score:
            best_score = score
            best_person = person
    return best_person, best_score

def run_probes(probes, gallery):
    y_true, y_pred, scores = [], [], []
    skipped = 0
    for true_person, img_path in tqdm(probes, desc="probes"):
        emb = image_to_embedding(img_path)
        if emb is None:
            skipped += 1
            continue
        pred_person, score = best_match(emb, gallery)
        y_true.append(true_person)
        y_pred.append(pred_person)
        scores.append(score)
    return np.array(y_true), np.array(y_pred), np.array(scores, np.float32), skippe
```

In [15]:
```python
val_y_true, val_y_pred, val_scores, val_skipped = run_probes(val_probes, val_galler
print("val evaluated:", len(val_scores), "skipped:", val_skipped)
```

```
val_is_correct = (val_y_true == val_y_pred).astype(np.int32)

def metrics_at_threshold(is_correct, scores, T):
    accept = (scores >= T).astype(np.int32)

    TP = int(np.sum((is_correct == 1) & (accept == 1)))
    FN = int(np.sum((is_correct == 1) & (accept == 0)))
    FP = int(np.sum((is_correct == 0) & (accept == 1)))
    TN = int(np.sum((is_correct == 0) & (accept == 0)))

    precision = TP / (TP + FP + 1e-12)
    recall    = TP / (TP + FN + 1e-12)
    f1        = 2 * precision * recall / (precision + recall + 1e-12)

    FAR = FP / (FP + TN + 1e-12)
    FRR = FN / (FN + TP + 1e-12)

    return {"T": float(T), "precision": precision, "recall": recall, "f1": f1, "FAR
            "TP": TP, "FP": FP, "TN": TN, "FN": FN}

Ts = np.linspace(0.2, 0.9, 200)

best = None
for T in Ts:
    m = metrics_at_threshold(val_is_correct, val_scores, T)
    if m["FAR"] <= 0.01:
        if best is None or m["f1"] > best["f1"]:
            best = m

if best is None:
    best = max((metrics_at_threshold(val_is_correct, val_scores, T) for T in Ts), k

best
```

```
probes:   0%|          | 0/489 [00:00<?, ?it/s]
val evaluated: 486 skipped: 3
```

Out[15]:
```
{'T': 0.5095477386934673,
 'precision': 0.9999999999999977,
 'recall': 0.9339019189765438,
 'f1': 0.9658213891946471,
 'FAR': 0.0,
 'FRR': 0.06609808102345402,
 'TP': 438,
 'FP': 0,
 'TN': 17,
 'FN': 31}
```

In [16]:
```
TEST_T = best["T"]
print("chosen threshold:", TEST_T)

test_y_true, test_y_pred, test_scores, test_skipped = run_probes(test_probes, test_
print("test evaluated:", len(test_scores), "skipped:", test_skipped)

test_is_correct = (test_y_true == test_y_pred).astype(np.int32)
```

```
test_metrics = metrics_at_threshold(test_is_correct, test_scores, TEST_T)
test_metrics
```

```
chosen threshold: 0.5095477386934673
probes:   0%|          | 0/479 [00:00<?, ?it/s]
test evaluated: 479 skipped: 0
```

Out[16]:
```
{'T': 0.5095477386934673,
 'precision': 0.9999999999999977,
 'recall': 0.9397849462365571,
 'f1': 0.9689578713963939,
 'FAR': 0.0,
 'FRR': 0.060215053763440725,
 'TP': 437,
 'FP': 0,
 'TN': 14,
 'FN': 28}
```

In [17]:
```
def check_access(img_path, gallery, T):
    emb = image_to_embedding(img_path)
    if emb is None:
        return {"status": "no_face", "authorized": False, "match": None, "score": N

    person, score = best_match(emb, gallery)
    authorized = (score >= T)
    return {"status": "ok", "authorized": bool(authorized), "match": person, "score

# Demo
if len(test_probes) > 0:
    true_person, probe_img = test_probes[0]
    out = check_access(probe_img, test_gallery, TEST_T)
    print("true:", true_person)
    print(out)
```

```
true: Nicole_Kidman
{'status': 'ok', 'authorized': True, 'match': 'Nicole_Kidman', 'score': 0.7035143375
396729}
```

In [ ]: